

Exp 10

→ Write a program to create a linked list & insert items from the user using menu driven

→ #include <stdio.h>
#include <stdlib.h>

```
struct node {  
    int data;  
    struct Node *next;  
};
```

```
void insertAtBeg (struct Node **head) {  
    struct Node *newNode = (struct Node *) malloc (sizeof  
    (struct Node));
```

```
    printf ("Enter value: ");  
    scanf ("%d", &newNode->data);
```

```
    newNode->next = *head  
    *head = newNode;  
}
```

```
void insertAtEnd (struct Node **head) {  
    struct Node *newNode = (struct Node *) malloc  
    (sizeof (struct Node));
```

```
printf ("Enter value: ");  
scanf ("%d", & newNode → data);  
newNode → next = NULL;
```

```
if (*head == NULL) {  
    *head = newNode;  
}
```

```
else {
```

```
    struct Node *temp = *head;  
    while (temp → next != NULL)  
        temp = temp → next;
```

```
    temp → next = newNode;
```

```
}
```

```
}
```

```
void insertAtPos (struct Node **head) {
```

```
    int pos;
```

```
    printf ("Enter position: ");
```

```
    scanf ("%d", &pos);
```

```
    struct Node *newNode = (struct Node *) malloc  
        (sizeof (struct Node));
```

```
    printf ("Enter value: ");
```

```
    scanf ("%d", &newNode → data);
```

```
    if (pos == 1) {
```

```
        newNode → next = *head;
```



```
* head = newNode;  
return;  
}
```

```
struct Node * temp = *head;  
for (int i = 1; i < pos - 1 && temp != NULL; i++)  
{  
    temp = temp -> next;  
}
```

```
if (temp == NULL) {  
    printf("Invalid position! \n");  
    free(newNode);  
    return;  
}
```

```
newNode -> next = temp -> next;  
temp -> next = newNode;  
}
```

```
void display (struct Node * head) {  
    if (head == NULL) {  
        printf("List is empty! \n");  
        return;  
    }  
}
```

```
struct Node * temp = head;  
printf("Linked List: ");
```

```
while (temp != NULL) {
    printf ("%d -> ", temp->data);
    temp = temp->next;
}
```

```
printf ("NULL\n");
}
```

```
int main () {
    struct Node *head = NULL;
    int choice
```

```
while (1) {
    printf ("\n ===== MENU ===== \n");
    printf ("1. Insert at beginning\n");
    printf ("2. Insert at End\n");
    printf ("3. Insert at any position\n");
    printf ("4. Display list\n");
    printf ("5. Exit\n");
    printf ("Enter choice : ");
    scanf ("%d", &choice);
```

```
switch (choice) {
    case 1 : insert At Beg (&head); break;
    case 2 : insert At End (&head); break;
    case 3 : insert At Pos (&head); break;
    case 4 : display (head); break;
    case 5 : exit (0);
```


EXPT.
NO.

NAME

```
default : printf ( "Invalid choice ! \n" );  
}  
}  
}
```

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

void insertAtBeg(struct Node **head) {
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));

    printf("Enter value: ");
    scanf("%d", &newNode->data);

    newNode->next = *head;
    *head = newNode;
}

void insertAtEnd(struct Node **head) {
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    printf("Enter value: ");
    scanf("%d", &newNode->data);
    newNode->next = NULL;

    if (*head == NULL) {
        *head = newNode;
    }
    else {
        struct Node *temp = *head;
        while (temp->next != NULL)
            temp = temp->next;

        temp->next = newNode;
    }
}
```

```
void insertAtPos(struct Node **head) {
    int pos;
    printf("Enter position: ");
    scanf("%d", &pos);

    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    printf("Enter value: ");
    scanf("%d", &newNode->data);

    if (pos == 1) {
        newNode->next = *head;
        *head = newNode;
        return;
    }

    struct Node *temp = *head;
    for (int i = 1; i < pos - 1 && temp != NULL; i++) {
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Invalid position!\n");
        free(newNode);
        return;
    }

    newNode->next = temp->next;
    temp->next = newNode;
}
```

```
void display(struct Node *head) {  
    if (head == NULL) {  
        printf("List is empty!\n");  
        return;  
    }  
  
    struct Node *temp = head;  
    printf("Linked List: ");  
    while (temp != NULL) {  
        printf("%d -> ", temp->data);  
        temp = temp->next;  
    }  
    printf("NULL\n");  
}
```



```
int main() {
    struct Node *head = NULL;
    int choice;

    while (1) {
        printf("\n==== MENU =====\n");
        printf("1. Insert at Beginning\n");
        printf("2. Insert at End\n");
        printf("3. Insert at Any Position\n");
        printf("4. Display List\n");
        printf("5. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: insertAtBeg(&head); break;
            case 2: insertAtEnd(&head); break;
            case 3: insertAtPos(&head); break;
            case 4: display(head); break;
            case 5: exit(0);
            default: printf("Invalid choice!\n");
        }
    }
}
```

```
PS C:\Users\sanch\OneDrive\Desktop\c programming> cd 'c:\Users\sanch\OneDrive\Desktop\C project\output'
PS C:\Users\sanch\OneDrive\Desktop\C project\output> & .\'Project.exe'
```

```
===== MENU =====
```

1. Insert at Beginning
2. Insert at End
3. Insert at Any Position
4. Display List
5. Exit

Enter choice: 4

List is empty!

```
===== MENU =====
```

1. Insert at Beginning
2. Insert at End
3. Insert at Any Position
4. Display List
5. Exit

Enter choice: 1

Enter value: 20

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

===== MENU =====

1. Insert at Beginning
2. Insert at End
3. Insert at Any Position
4. Display List
5. Exit

Enter choice: 1

Enter value: 40

===== MENU =====

1. Insert at Beginning
2. Insert at End
3. Insert at Any Position
4. Display List
5. Exit

Enter choice: 2

Enter value: 100

===== MENU =====

1. Insert at Beginning
2. Insert at End
3. Insert at Any Position
4. Display List
5. Exit

Enter choice: 2

Enter value: 99

===== MENU =====

1. Insert at Beginning
2. Insert at End
3. Insert at Any Position
4. Display List
5. Exit

Enter choice: 1

Enter value: 43

1. Insert at Beginning
2. Insert at End
3. Insert at Any Position
4. Display List
5. Exit

Enter choice: 3

Enter position: 4

Enter value: 55

===== MENU =====

1. Insert at Beginning
2. Insert at End
3. Insert at Any Position
4. Display List
5. Exit

Enter choice: 4

Linked List: 43 -> 40 -> 20 -> 55 -> 100 -> 99 -> NULL

===== MENU =====

1. Insert at Beginning
2. Insert at End
3. Insert at Any Position
4. Display List
5. Exit

Enter choice: 5

PS C:\Users\sanch\OneDrive\Desktop\C project\output>